



Dynamic Hashing



Dynamic Hashing

❑ Example-3

The following key values are organized in an extendible hashing technique.

1, 3, 5, 8, 9, 12, 17, 28

Show the extendible hash structure for this file if the hash function is $h(x) = x \bmod 8$ and buckets can hold three records. Show how extendible hash structure changes as the result of each of the following operations:

- i. Insert 2
- ii. Insert 24
- iii. Delete 5
- iv. Delete 12



Dynamic Hashing

❑ Example

Solution: Convert the results of the hash function into binary representation:

Data	$h(x) = x \bmod 8$	Binary form
1	$1 \bmod 8 = 1$	001
3	$3 \bmod 8 = 3$	011
5	$5 \bmod 8 = 5$	101
8	$8 \bmod 8 = 0$	000
9	$9 \bmod 8 = 1$	001
12	$12 \bmod 8 = 4$	100
17	$17 \bmod 8 = 1$	001
28	$28 \bmod 8 = 4$	100

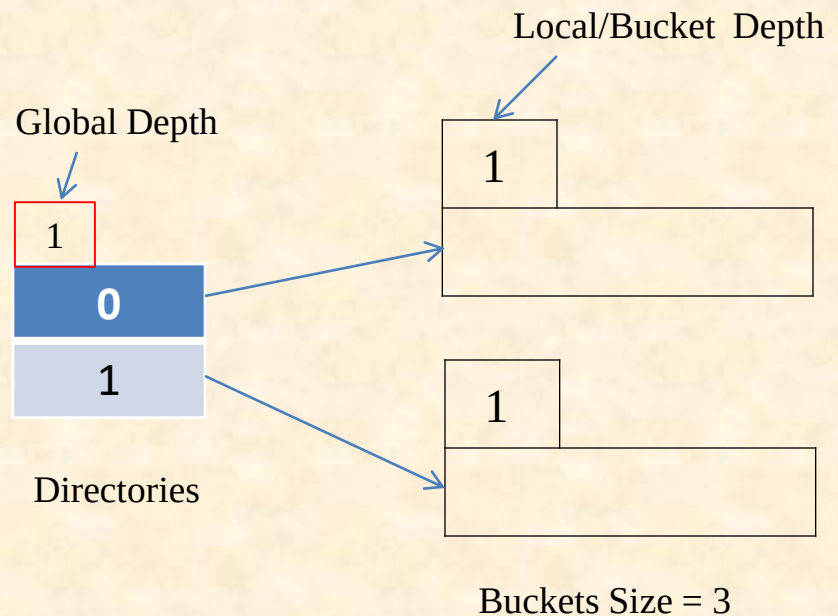


Dynamic Hashing

❑ Example

Solution: Initialize the hash table with two initial directories and buckets. Set the global depth and bucket depth to 1

Data	Binary form
1	001
3	011
5	101
8	000
9	001
12	100
17	001
28	100



Initial Hash structure

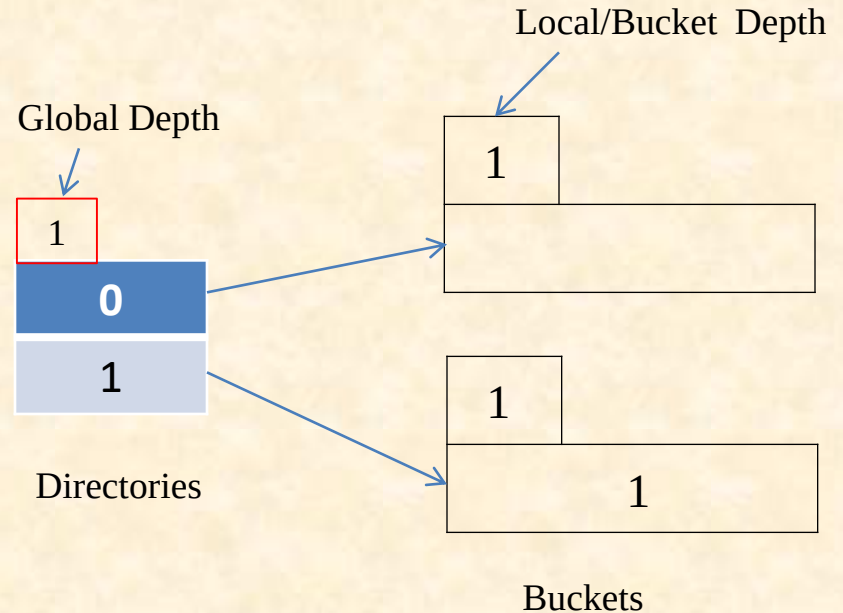


Dynamic Hashing

❑ Example

Solution: Inserting 1 (00**1**). Global-depth is 1. The hash function returns 1 LSB of 00**1** which is 1. Hence, 1 is mapped to the directory with id=1.

Data	Binary form
1	00 1
3	011
5	101
8	000
9	001
12	100
17	001
28	100



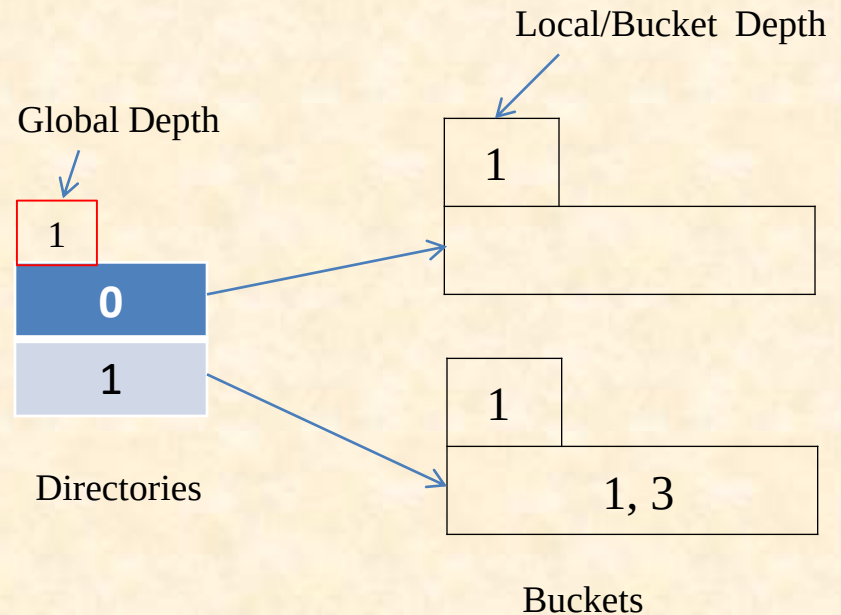


Dynamic Hashing

❑ Example

Solution: Inserting 3 (01**1**). The hash function returns 1 LSB of 01**1** which is 1. Hence, 3 is mapped to the directory with id=1.

Data	Binary form
1	001
3	01 1
5	101
8	000
9	001
12	100
17	001
28	100



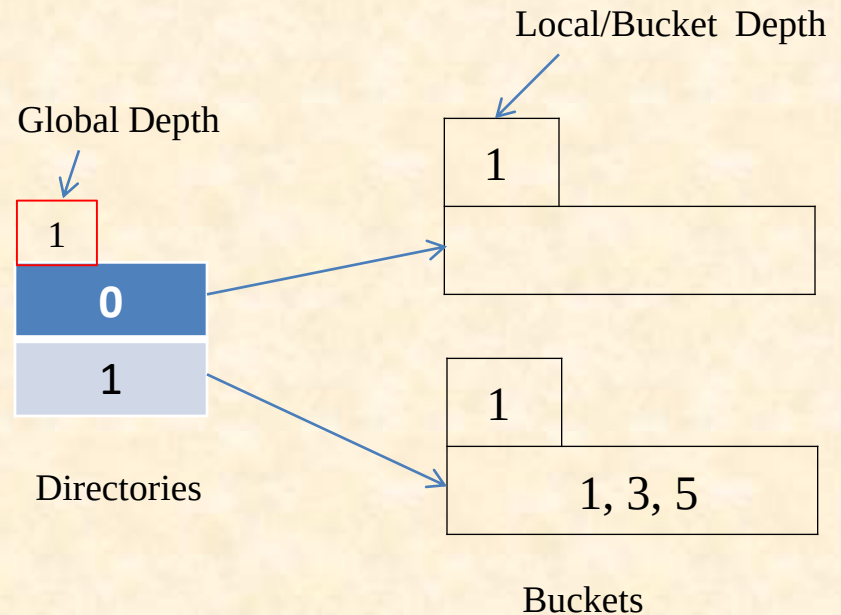


Dynamic Hashing

❑ Example

Solution: Inserting 5 (**101**). Add it to the "1" ID bucket as the hash function returns 1 LSB of 10**1** which is 1.

Data	Binary form
1	001
3	011
5	10 1
8	000
9	001
12	100
17	001
28	100



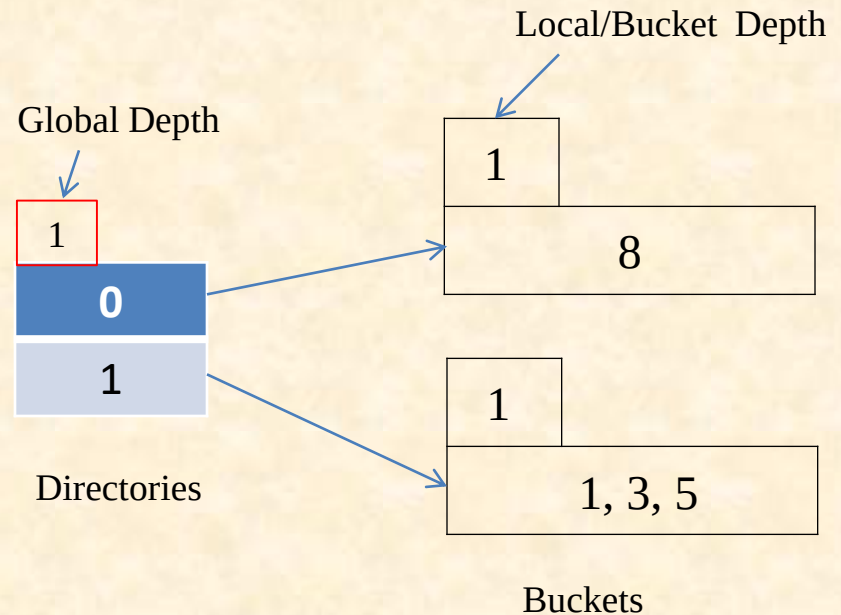


Dynamic Hashing

❑ Example

Solution: Inserting 8 (00**0**). Add it to the “0” ID bucket as the hash function returns 1 LSB of 00**0** which is 0.

Data	Binary form
1	001
3	011
5	101
8	00 0
9	001
12	100
17	001
28	100



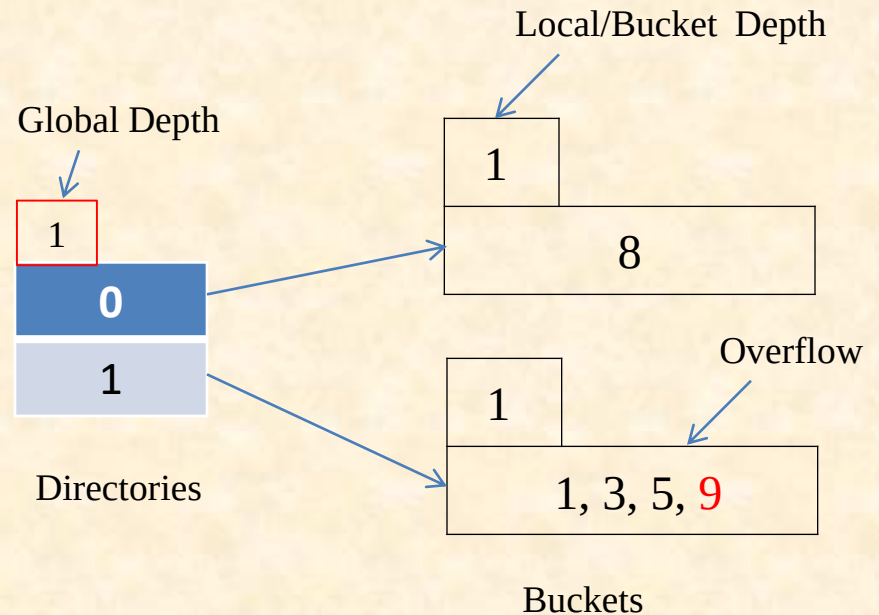


Dynamic Hashing

❑ Example

Solution: Inserting 9 (00**1**). Adding it to the “1” ID bucket makes it overflow.

Data	Binary form
1	001
3	011
5	101
8	000
9	00 1
12	100
17	001
28	100



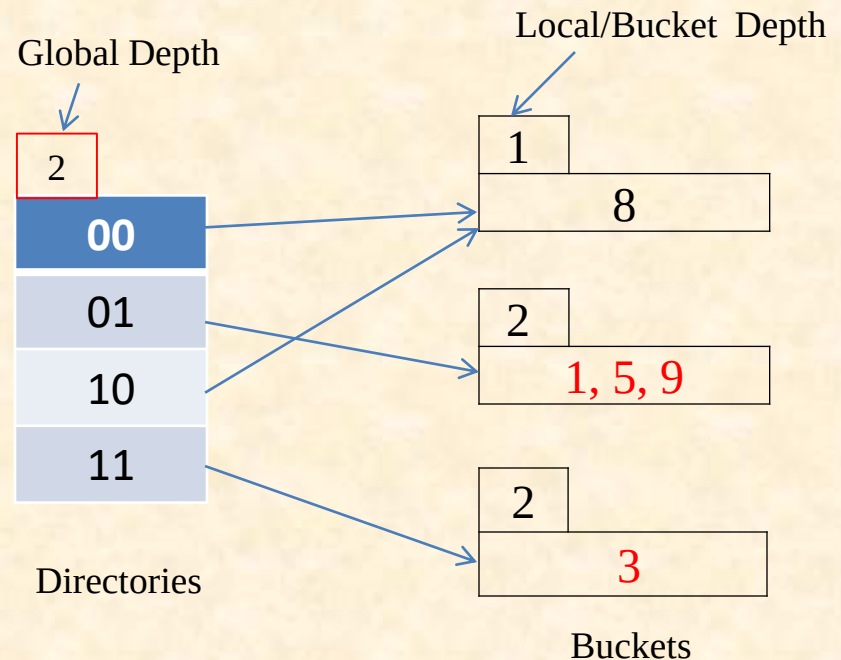


Dynamic Hashing

❑ Example

Solution: Since Local Depth = Global Depth, the bucket splits and directory expansion takes place. Rehash the numbers present in the overflowing bucket after splitting the bucket. Global depth is incremented by 1. Now, the global depth is 2. Hence, 1, 3, 5 and 9 are now rehashed w. r. t 2 LSBs.

Data	Binary form
1	001
3	011
5	101
8	000
9	001
12	100
17	001
28	100



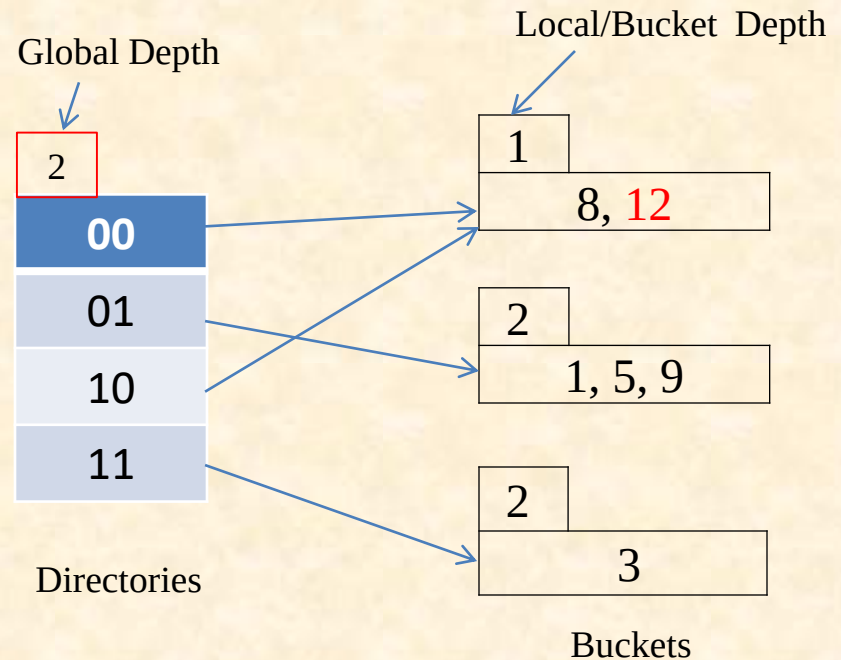


Dynamic Hashing

❑ Example

Solution: Inserting 12 (1**00**). Adding it to the "00" ID bucket.

Data	Binary form
1	001
3	011
5	101
8	000
9	001
12	1 00
17	001
28	100



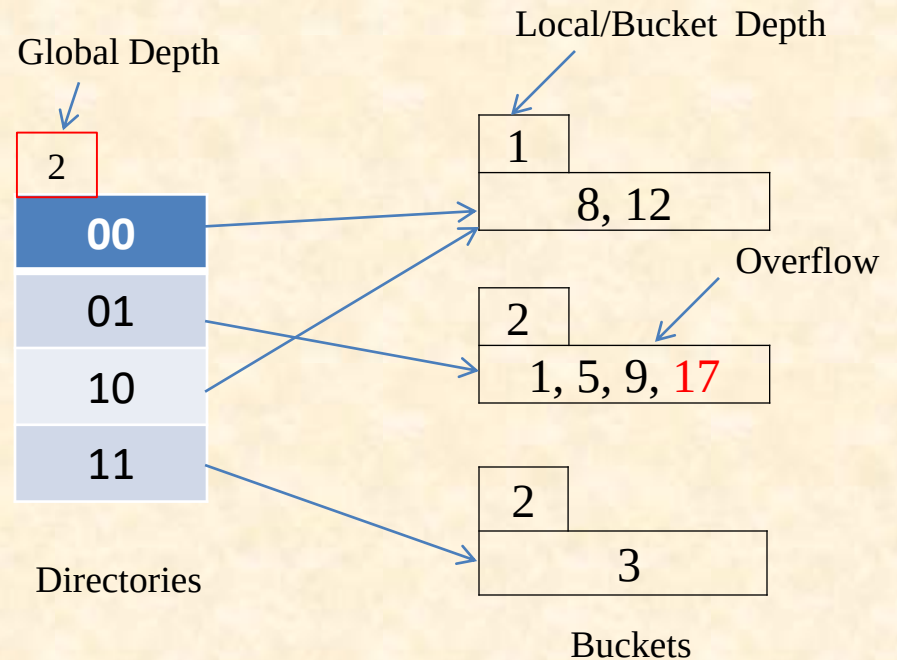


Dynamic Hashing

❑ Example

Solution: Inserting 17 (0**01**). Adding it to the "01" ID bucket makes it overflow.

Data	Binary form
1	001
3	011
5	101
8	000
9	001
12	100
17	001
28	100



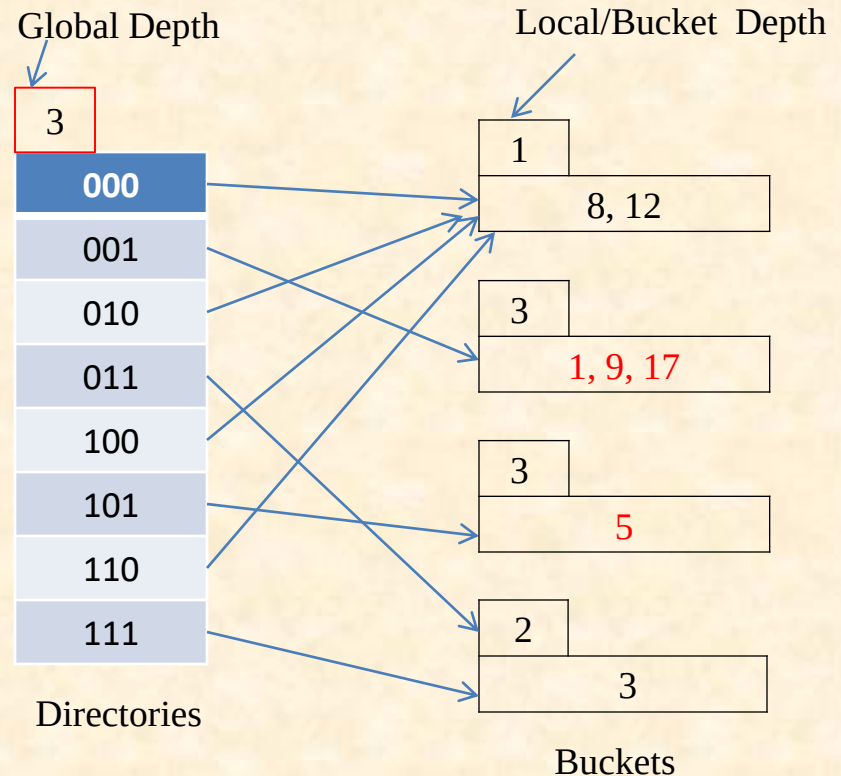


Dynamic Hashing

❑ Example

Solution: Since Local Depth = Global Depth, the bucket splits and directory expansion takes place. Rehash the numbers present in the overflowing bucket after splitting the bucket. Global depth is incremented by 1. Now, the global depth is 3. Hence, 1, 5, 9 and 17 are now rehashed w. r. t 3 LSBs.

Data	Binary form
1	001
3	011
5	101
8	000
9	001
12	100
17	001
28	100



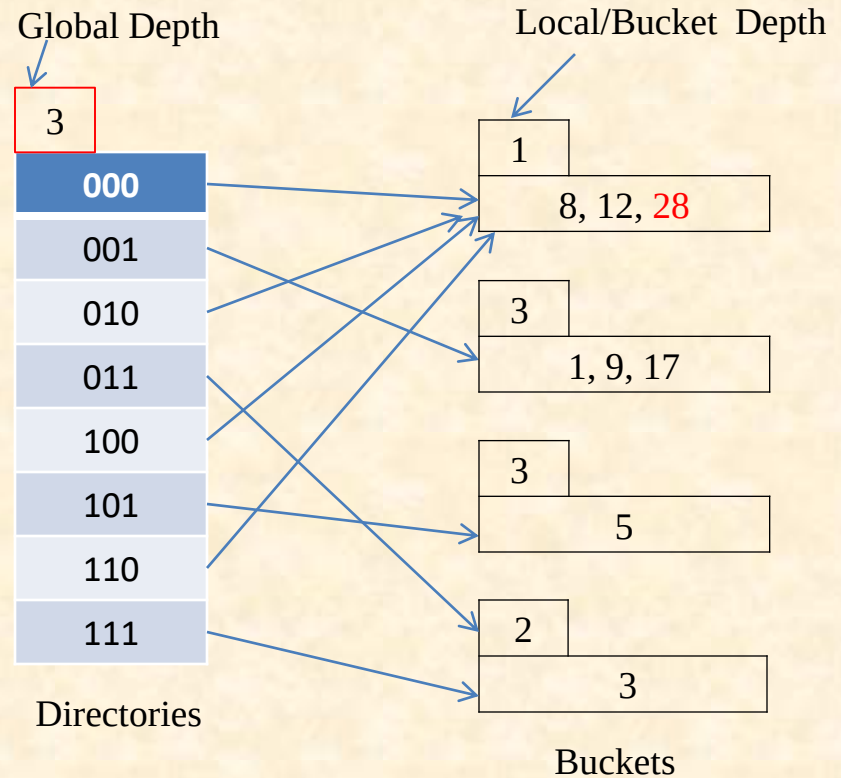


Dynamic Hashing

❑ Example

Solution: Inserting 28 (**100**). Adding it to the “100” ID bucket.

Data	Binary form
1	001
3	011
5	101
8	000
9	001
12	100
17	001
28	100

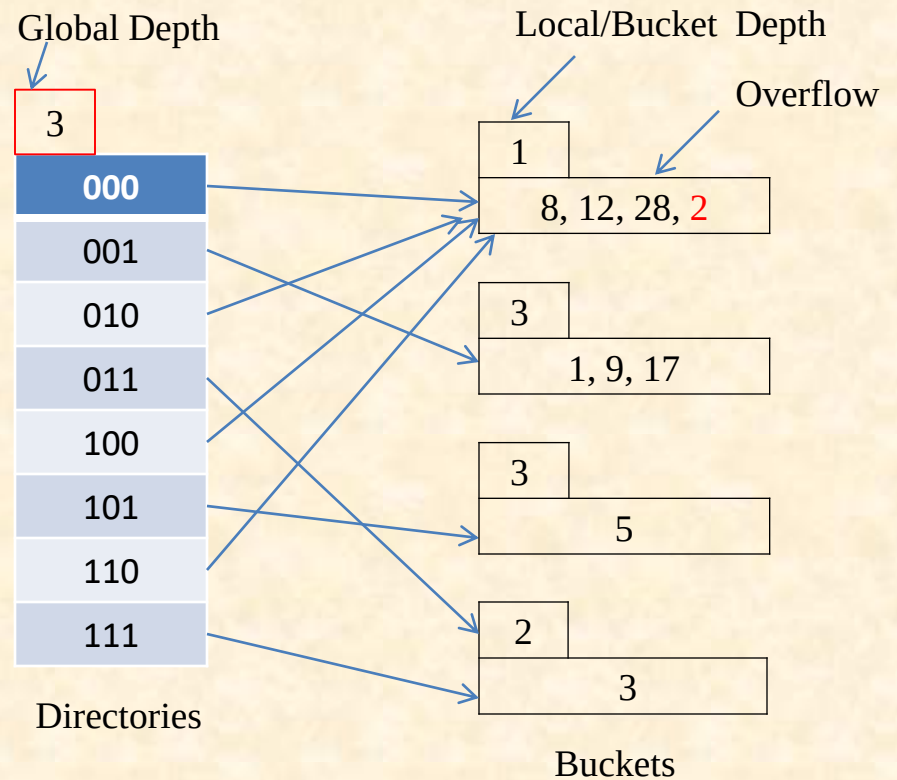




Dynamic Hashing

❑ Example

Solution: i) Insert 2 (**010**). Adding it to the “010” ID bucket makes it overflow.





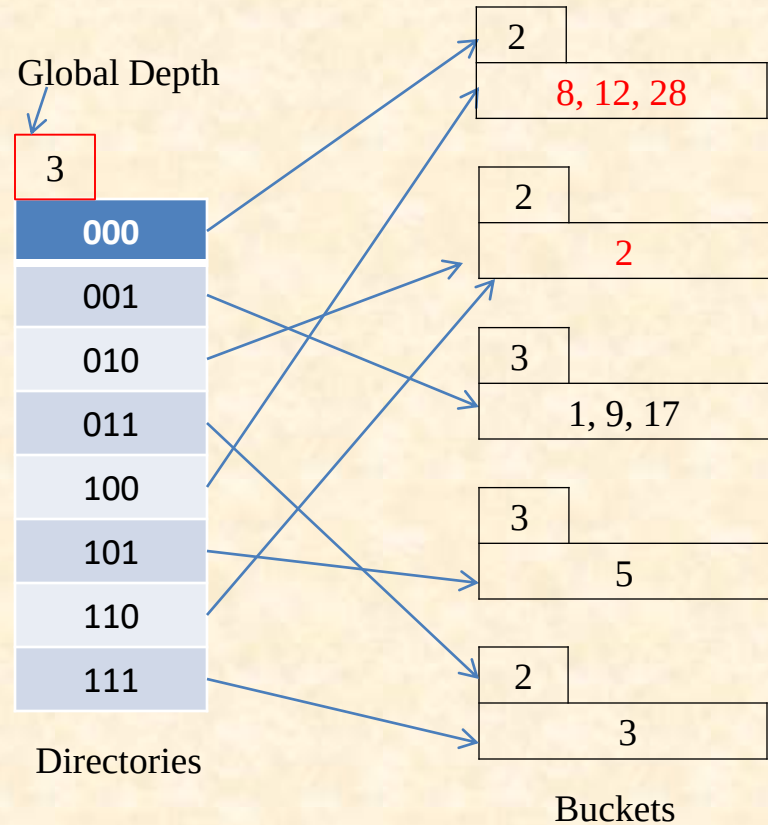
Dynamic Hashing

❑ Example

Solution: i) Insert 2 (**010**).

Here Bucket depth is less than global depth. So, only the bucket split is executed. Elements (8, 12, 28, 2) are re-hashed according to the global depth, which is still 3.

Data	Binary form
1	001
3	011
5	101
8	000
9	001
12	100
17	001
28	100

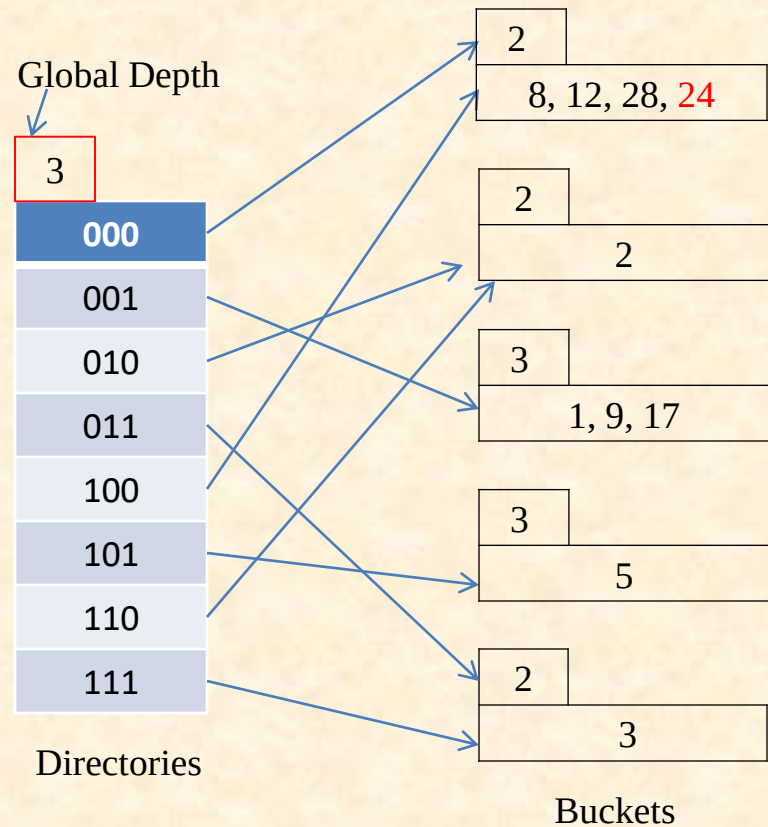




Dynamic Hashing

❑ Example

Solution: ii) Insert 24 (**000**). Adding it to the “000” ID bucket makes it overflow.





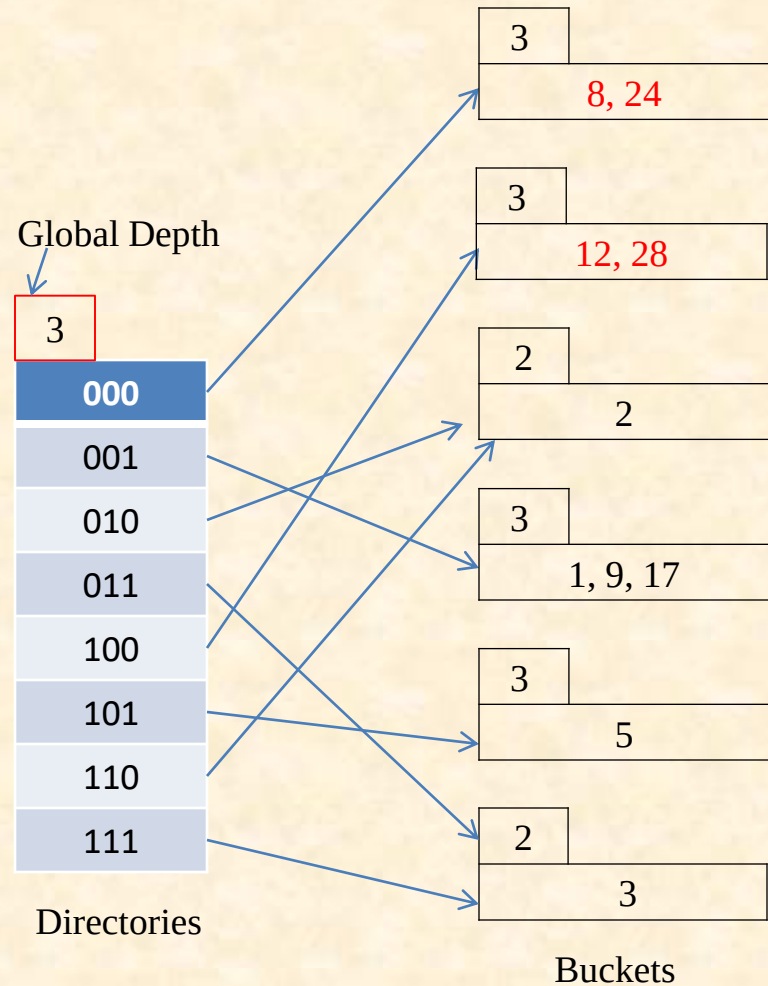
Dynamic Hashing

❑ Example

Solution: ii) Insert 24 (**000**).

Here Bucket depth is less than global depth. So, only the bucket split is executed. Elements (8, 12, 28, 24) are re-hashed according to the global depth, which is still 3.

Data	Binary form
1	001
3	011
5	101
8	000
9	001
12	100
17	001
28	100





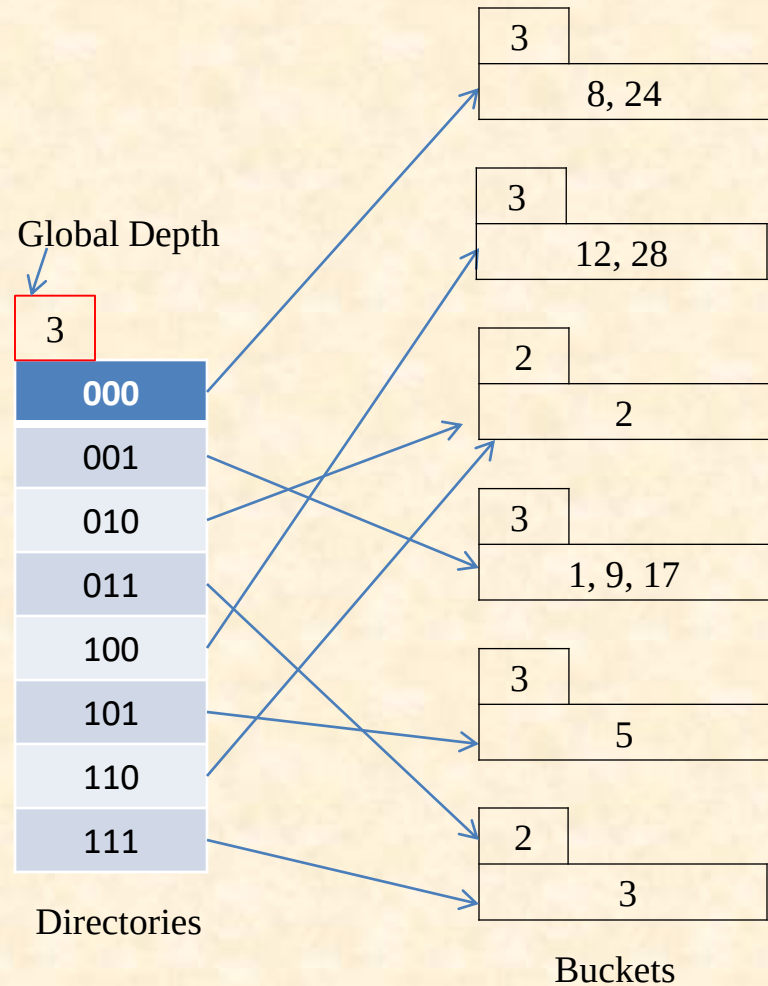
Dynamic Hashing

❑ Example

Solution: iii) Delete 5

On deleting 5, we get one bucket pointed by 101 as empty. This will also result in reducing the local depth of the bucket pointed by 001.

Data	Binary form
1	001
3	011
5	101
8	000
9	001
12	100
17	001
28	100





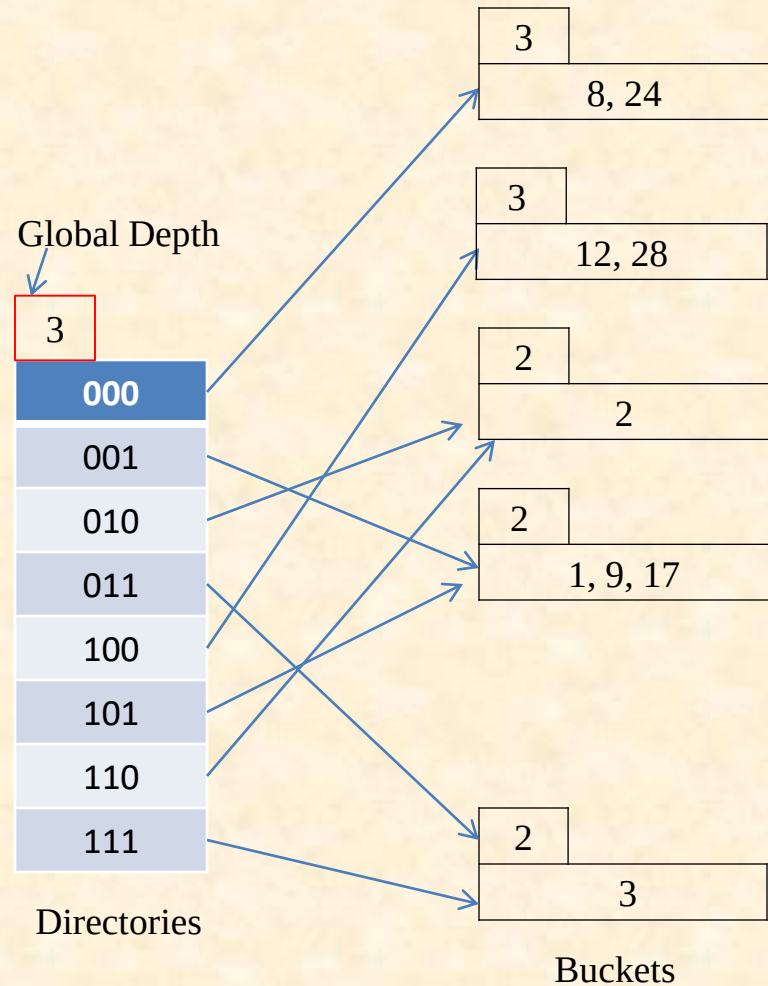
Dynamic Hashing

❑ Example

Solution: iii) Delete 5

Hence, we get-

Data	Binary form
1	001
3	011
5	101
8	000
9	001
12	100
17	001
28	100





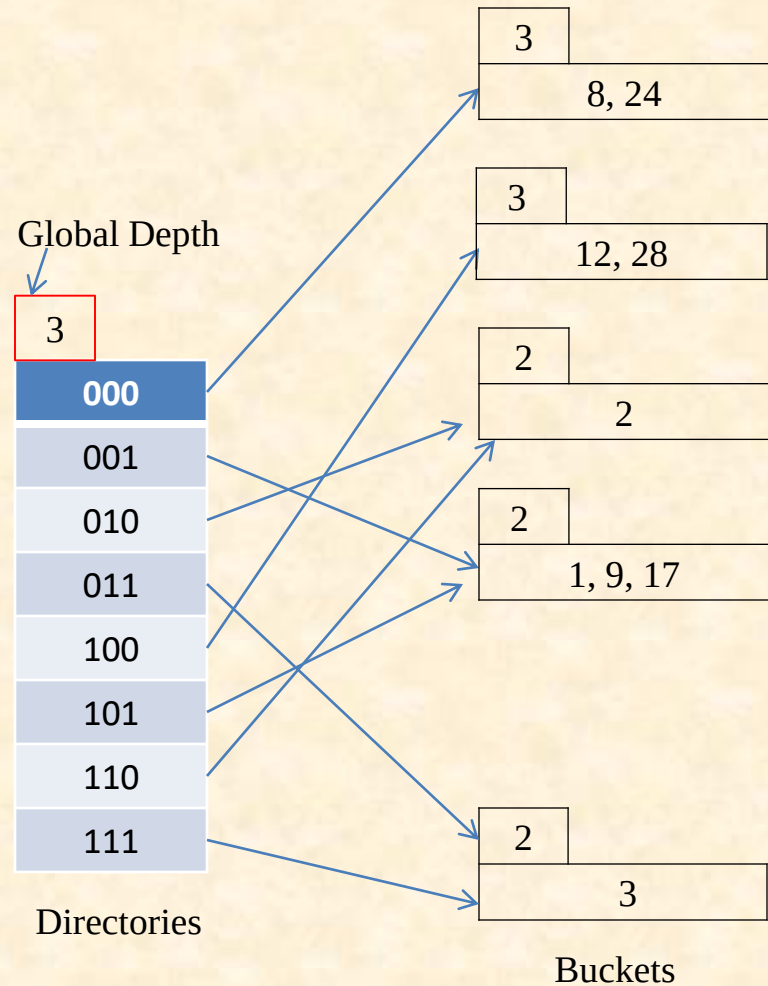
Dynamic Hashing

❑ Example

Solution: iv) Delete 12.

We will simply delete 12 from the corresponding bucket there can not be any merging of buckets on deletion.

Data	Binary form
1	001
3	011
5	101
8	000
9	001
12	100
17	001
28	100





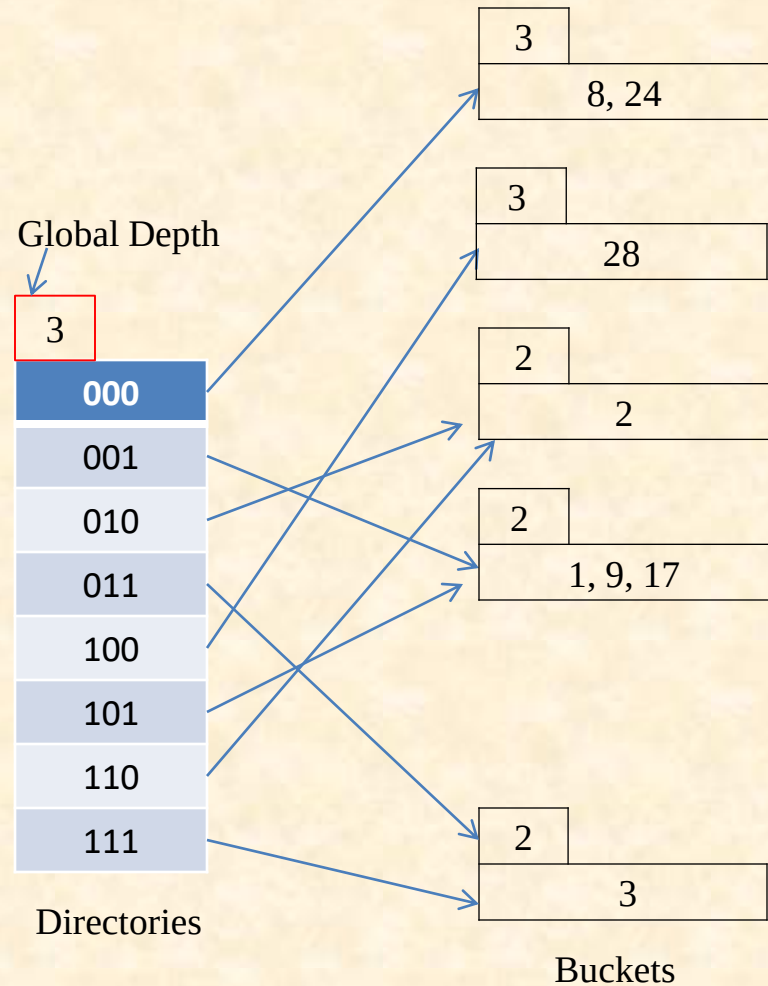
Dynamic Hashing

❑ Example

Solution: iv) Delete 12.

The result of deletion 12 is as given below

Data	Binary form
1	001
3	011
5	101
8	000
9	001
12	100
17	001
28	100





Dynamic Hashing

❑ Example Static Hashing VS Dynamic Hashing

Static Hashing:

- The number of buckets is fixed.
- Chaining is used.
- Open hashing and closed hashing are forms of static hashing.
- Space overhead is more.
- As file grows, the performance of static hashing decreases.
- The bucket address table is not required.
- The bucket is directly accessed.



Dynamic Hashing

❑ Example Static Hashing VS Dynamic Hashing

Dynamic Hashing:

- The number of buckets is not fixed.
- There is no need of chaining.
- Extendible hashing and Linear hashing are forms of dynamic hashing.
- Minimum space overhead due to dynamic nature.
- There is no degradation in performance while the file grows.
- The bucket address table is required.
- The bucket address table is used to access the bucket.



Dynamic Hashing

- ❑ **Why is a hash structure not the best choice for a search key on which range queries are likely?**

A range query cannot be answered efficiently using a hash index, we will have to read all the buckets. This is because key values in the range do not occupy consecutive locations in the buckets, they are distributed uniformly and randomly throughout all the buckets.



Dynamic Hashing

□ Example A

Suppose that we are using extendable hashing on a file that contains records with the following search-key values:

2, 3, 5, 7, 11, 17, 19, 23, 29, 31

Show the extendable hash structure for this file if the hash function is $h(x) = x \bmod 8$ and buckets can hold three records.

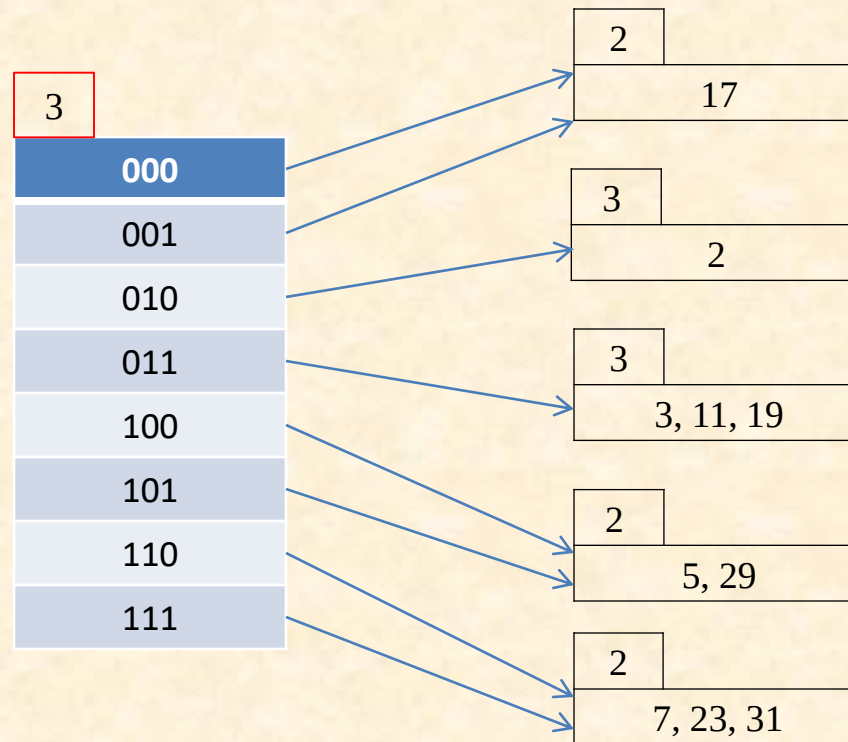


Dynamic Hashing

❑ Example A

Solution:

Final extendable hash structure:





Dynamic Hashing

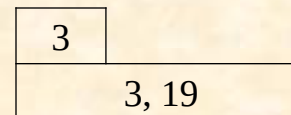
❑ Example B

Show how the extendable hash structure of Example A changes as the result of each of the following steps:

- Delete 11.
- Delete 31.
- Insert 1.
- Insert 15.

Solution: a. Delete 11.

Delete 11 from the third bucket.





Dynamic Hashing

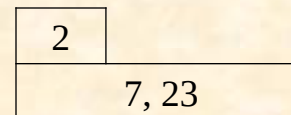
❑ Example

Show how the extendable hash structure of Q. 26 changes as the result of each of the following steps:

- Delete 11.
- Delete 31.
- Insert 1.
- Insert 15.

Solution: b. Delete 31.

Delete 31 from the Last bucket.





Dynamic Hashing

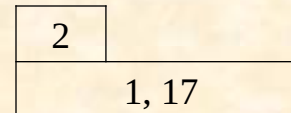
❑ Example

Show how the extendable hash structure of Q. 26 changes as the result of each of the following steps:

- Delete 11.
- Delete 31.
- Insert 1.
- Insert 15.

Solution: c. Insert 1.

Insert 1 into the first bucket.





Dynamic Hashing

❑ Example

Show how the extendable hash structure of Q. 26 changes as the result of each of the following steps:

- Delete 11.
- Delete 31.
- Insert 1.
- Insert 15.

Solution: d. Insert 15.

Insert 15 into the last bucket.

2
7, 15, 23